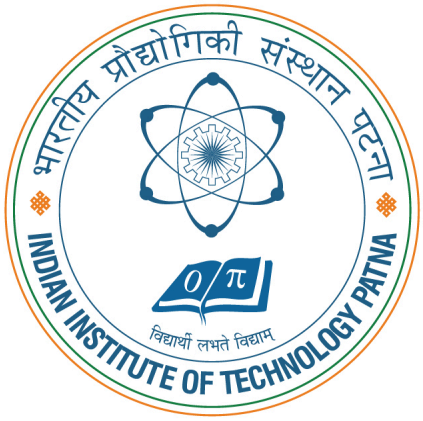
Computer Science & Engineering

Indian Institute of Technology Patna



PROJECT REPORT

C-Grade

Automated C-Code Grading System

Submitted by

Name	Roll Number
Manas Gogna	2611CS10
Husain Kanchwala	2611CS04
Jay Agrawal	2611AI09
Adnan Hasan	2611MC09

Submitted as part of the Mini Project / Lab Work

Table of Content

1. What We Are Building and Why (Project Overview)..... 3
2. System Architecture (Streamlit Frontend + Python Backend)..... 4
2.1 How the Two Web Portals Work..... 4
2.2 Flowchart 1: Web Portal & Backend Architecture..... 5
3. End-to-End 3-Stage Grading Pipeline..... 6
3.1 Flowchart 2: How a Submission Travels Through the 3 Stages..... 6
4. The 9 Specialist Analyzers & Counter-Example Generator..... 7
Metric..... 7
Module..... 7
How It Works (Tools & Method)..... 7
How We Keep It Reproducible..... 7
5. Testing and Quality Assurance..... 8
5.1 Independent Analyzer Testing via test_tool.py..... 8
5.2 End-to-End Pipeline Testing..... 9
5.3 Reproducibility Verification..... 9
5.4 Parallel Execution Testing (Planned for Week 3)..... 9
6. Real-Life Usefulness..... 10
7. Workload Distribution (4 Team Members)..... 10
Member & Roll No..... 10
Role..... 10
Assigned Modules..... 10
Core Responsibilities..... 10
8. Progress So Far & 3-Week Timeline..... 11
8.1 Progress So Far (Week 2 Update)..... 11
8.2 Our 3-Week Plan..... 11
Week..... 12
Milestone Focus..... 12
Key Deliverables..... 12
Lab Checkpoint..... 12

1. What We Are Building and Why (Project Overview)

Grading C programs by hand for a full class is slow, repetitive, and often uneven. Different evaluators check different things, and a single student program with an infinite loop or a bad pointer can freeze a basic grading script.

To solve this, we are building **C-Grade (AutoGrader)** — a complete, self-running grading system with a **Streamlit Web App (ui.py)** on the frontend and a **Python 3.11 Backend (agent.py)**, powered offline by a local AI model via **Ollama (qwen3.5-2b)**.

Our system is built on three clear promises:

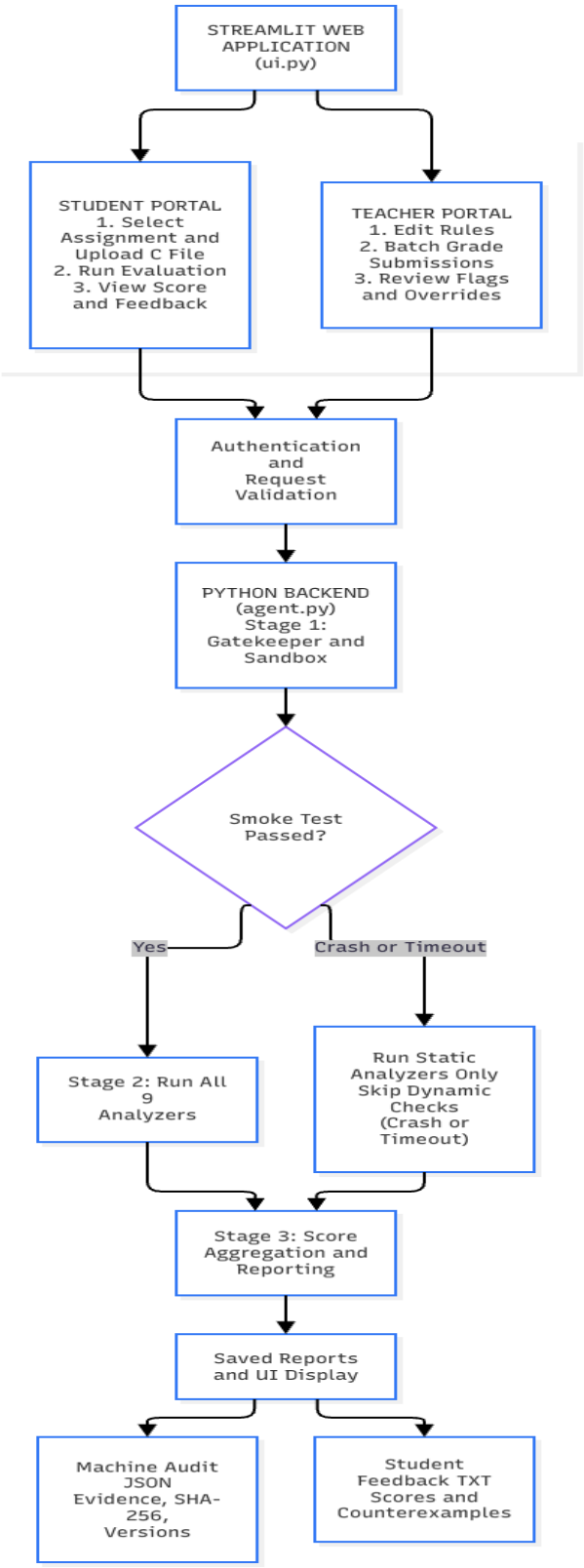
1. **Same Code + Same Rules = Similar Grade Every Time:**
   Running the same C file with the same rules.json file always gives a similar score and feedback. We do this by using fixed test cases, counting CPU instructions (perf) instead of shaky wall-clock timers, and locking our local Ollama AI model to temperature = 0 and seed = 42.
2. **Rules Live Outside the Code (rules.json):**
   Teachers never need to edit Python code to change grading rules. Every metric, weight, threshold, timeout, and memory limit lives in rules.json and can be edited directly from the Streamlit Teacher Portal.
3. **Safe Sandbox Execution & Real Counter-Examples:**
   Every student binary runs inside a strict sandbox (1s timeout, 64 MB memory limit) so crashes and infinite loops are caught safely. When a check fails, the grader outputs a concrete counter-example showing the exact **INPUT**, **EXPECTED** output, what the program actually **GOT**, and **WHY** it failed.

2. System Architecture (Streamlit Frontend + Python Backend)

2.1 How the Two Web Portals Work

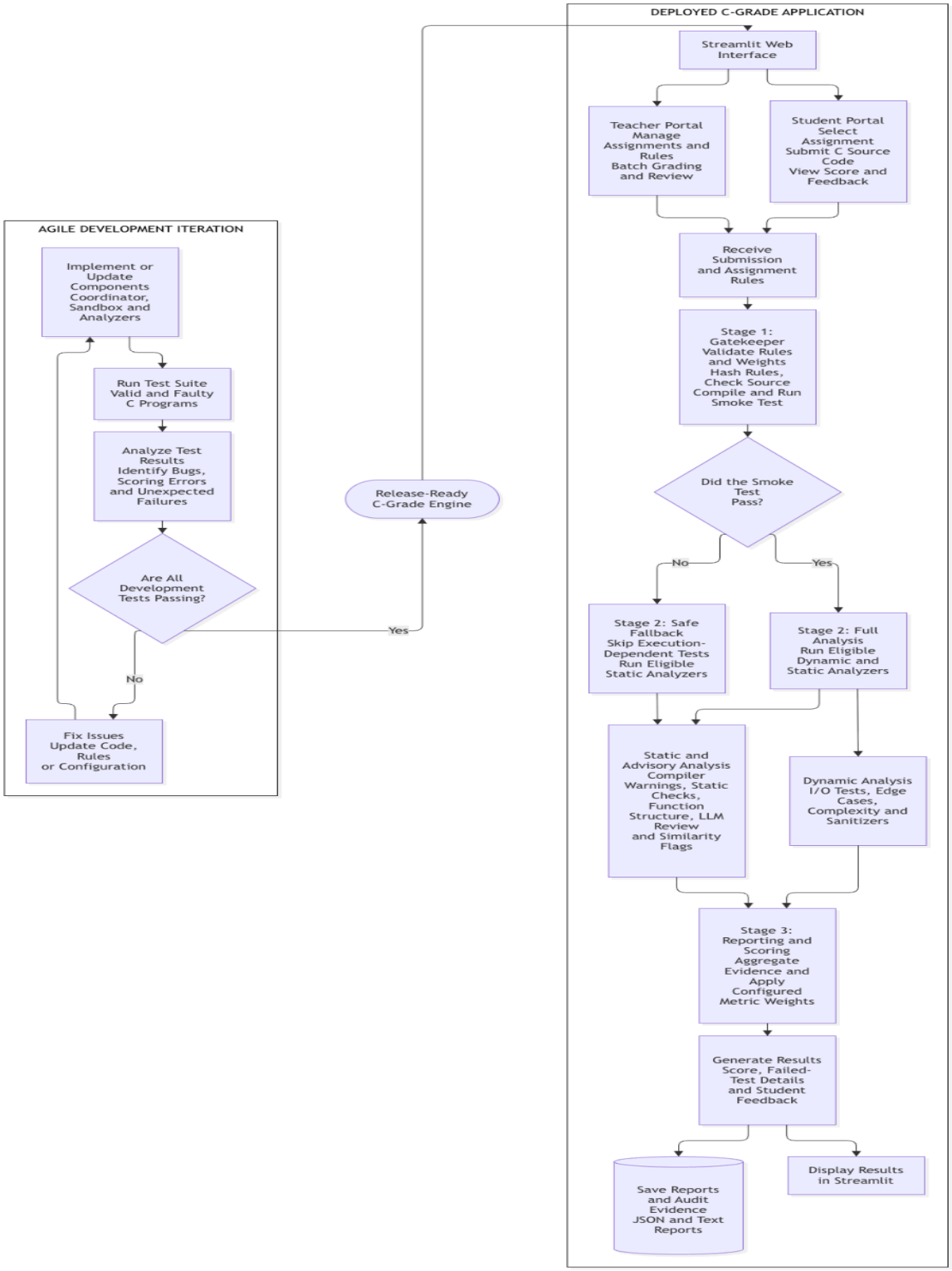
- **Student Portal (Streamlit):**
  Students select an active assignment, upload their .c file (such as add.c), click **Evaluate My Code**, and immediately see their overall grade, a breakdown across all 9 metrics, compiler warnings, memory leak diagnostics, and counter-examples showing how to fix their bugs.
- **Teacher Portal (Streamlit):**
  Instructors visually configure rules.json (weights, limits, and thresholds), trigger **Batch Grading** across an entire folder of student submissions, check the **Plagiarism Triage Dashboard** for flagged pairs (similarity >= 0.85), inspect machine JSON logs, and override grades with audit logging.

2.2 Flowchart 1: Web Portal & Backend Architecture



3. End-to-End 3-Stage Grading Pipeline

3.1 Flowchart 2: How a Submission Travels Through the 3 Stages



4. The 9 Specialist Analyzers & Counter-Example Generator

Every analyzer uses a single shared Python function format: `evaluate(source_path, binary_path, params, limits) -> dict`

Metric	Module	How It Works (Tools & Method)	How We Keep It Reproducible
correctness	`io_tests.py`	Runs compiled binary on hidden .in/.out files in testset_dir.	Fixed inputs & 1s/64MB sandbox caps.
syntax	`compiler_warnings.py`	Parses gcc -fsyntax-only -Wall -Wextra -std=c11 warnings.	Pinned GCC version & fixed deduction.
structure	`static_analysis.py`	Uses cppcheck + lizard to check cyclomatic complexity & function length.	Deterministic AST rules.
modularity	`function_split.py`	Uses lizard AST check to count helper functions against min_functions.	Deterministic AST function count.
logical	`edge_case_runner.py`	Combines C boundary values (0, negatives, INT_MAX, INT_MIN) + Ollama (qwen3.5-2b) and checks against reference oracle.c.	Fixed templates + Ollama temperature=0, seed=42.
complexity	`complexity_estimator.py`	Runs scaled input sizes under perf stat instruction counts and fits complexity curve.	Hardware CPU instruction counts avoid OS timing noise.

Metric	Module	How It Works (Tools & Method)	How We Keep It Reproducible
memory_safety	`sanitizer_run.py`	Recompiles with -fsanitize=address,undefined (ASan/UBSan) and runs valgrind.	Deterministic sanitizer exit codes.
readability	`llm_review.py`	Local Ollama (qwen3.5-2b on port 11434) scores naming, comments, and formatting.	format="json", temperature=0, seed=42.
originality	`similarity_flag.py`	Normalizes C tokens and runs k-gram winnowing against corpus_dir submissions.	Exact Jaccard token similarity math.

5. Testing and Quality Assurance

To ensure the reliability, accuracy, and robustness of the C-Grade evaluator, comprehensive testing protocols were established. We are making use of an Agile iterative methodology in our testing and observing loops. We verify the performance of the system from individual, isolated modules up to the complete end-to-end pipeline.

5.1 Independent Analyzer Testing via test_tool.py

Before integrating the analyzers into the main application, each module is rigorously tested in isolation using a dedicated CLI harness: test_tool.py. By passing the specific C source file, the target analyzer, and the active rules.json file, we can debug and validate the behavior of each specialized component.

- **I/O Black-Box & Auto-Generator (io_tests):**
  Command: python test_tool.py --source submissions/add.c --analyzer io_tests --rules rules/add_rules.json
- **Adversarial Edge Cases (edge_case_runner):**
  Command: python test_tool.py --source submissions/add.c --analyzer edge_case_runner --rules rules/add_rules.json

- **Memory Safety (sanitizer_run):**
  Command: python test_tool.py --source submissions/add.c --analyzer sanitizer_run --rules rules/add_rules.json
- **Asymptotic Profiling (complexity_estimator):**
  Command: python test_tool.py --source submissions/add.c --analyzer complexity_estimator --rules rules/add_rules.json
- **Local LLM Review (llm_review):**
  Command: python test_tool.py --source submissions/add.c --analyzer llm_review --rules rules/add_rules.json

5.2 End-to-End Pipeline Testing

While we use test_tool.py to independently test every analyzer, we use our developed Streamlit UI to test the project end-to-end. To test end-to-end, we are displaying the output of every analyzer on the UI, so that we can observe and understand the issues if there are any. The test cases are also displayed on the UI - so that we can efficiently debug and fix the issues that arise, thus improving our project.

5.3 Reproducibility Verification

A core tenet of the project is grading consistency. We test for reproducibility by submitting the same .c file multiple times and verifying identical scores down to the decimal, confirming that llm_review maintains deterministic outputs (locked at temperature = 0 and seed = 42), and ensuring complexity_estimator leverages black-box massive scale timing to eliminate hardware jitter.

5.4 Parallel Execution Testing (Planned for Week 3)

Currently, the pipeline executes sequentially to verify accuracy. In Week 3, we will implement parallel dispatch using ThreadPoolExecutor. Testing will focus on thread safety and resource management.

---

6. Real-Life Usefulness

1. **Automated Grading for Large University Courses (CS101 Labs):**
   Grades 400+ student C submissions in minutes with uniform feedback across all 9 metrics instead of slow, uneven manual TA grading.
2. **Technical Hiring & Coding Interview Screens:**
   Evaluates 500 job applicants not just on basic output, but on memory leaks (valgrind/ASan), time complexity (perf), and edge-case safety, giving recruiters a clear per-metric justification for every candidate.
3. **Plagiarism & AI-Copy Triage for Instructors:**
   Flags only submission pairs whose structural token similarity crosses 0.85, letting instructors review 10–15 suspicious pairs instead of manually checking 300 files.
4. **Self-Service Debugging Lab for Students:**
   Students can upload drafts to the Streamlit Student Portal before the deadline to catch memory bugs, compiler warnings, and corner-case failures on their own.

7. Workload Distribution (4 Team Members)

Member & Roll No.	Role	Assigned Modules	Core Responsibilities
Manas Gogna *(2611CS10)*	Coordinator & Sandbox Lead	`agent.py`, `core/first_pass.py`, `core/sandbox.py`, `test_tool.py`	Main orchestrator, JSON rule parser & SHA-256 hashing, Stage 1 Gatekeeper, 1s/64MB sandbox, and parallel ThreadPoolExecutor dispatch.
Husain Kanchwala *(2611CS04)*	Dynamic & Profiling Lead	`io_tests.py`, `edge_case_runner.py`, `sanitizer_run.py`,	Hidden I/O runner, hybrid edge-case & counter-example generator with oracle.c, ASan/UBSan/Valgrind

Member & Roll No.	Role	Assigned Modules	Core Responsibilities
		`complexity_estimator.py`	runner, and perf complexity estimator.
Jay Agrawal *(2611AI09)*	Static Analysis & Originality Lead	`compiler_warnings.py`, `static_analysis.py`, `function_split.py`, `similarity_flag.py`	GCC warning parser, cppcheck/lizard integration, AST helper modularity check, and k-gram token-winnowing plagiarism detector.
Adnan Hasan *(2611MC09)*	Local LLM, Reporting & UI Lead	`llm_review.py`, `core/reporter.py`, `ui.py (Streamlit)`, `rules/`	Local Ollama (qwen3.5-2b) setup, failure explanation synthesis, .json/.txt report generation, and Student/Teacher Streamlit portals.

8. Progress So Far & 3-Week Timeline

8.1 Progress So Far (Week 2 Update)

We have successfully built and tested all 9 core analyzers. The Streamlit web app is fully up and running for both students and teachers. We have also confirmed that our grading is 100% consistent (the same code always gets the same score). Right now, the system runs sequentially, which is what we will demo this week.

8.2 Our 3-Week Plan

- **Week 1 (Oct 16 – Oct 22):** Set up the core system, GitHub, the security gatekeeper, the safe sandbox, and the first few analyzers. (Completed)
- **Week 2 (Oct 23 – Oct 29):** Finish all 9 analyzers, build the web app, and verify that grading is consistent. (In Progress - Demoing this week)
- **Week 3 (Oct 30 – Nov 5):** Make the analyzers run all at once (in parallel) to speed things up, add final polish to the web app, and record our final YouTube demo video.

Week	Milestone Focus	Key Deliverables	Lab Checkpoint
Week 1 *(Oct 16 – Oct 22)*	Core Skeleton & Static/Dynamic Base	Submit agentic01 report; initialize GitHub repo (c-grade); build first_pass.py, sandbox.py, test_tool.py, io_tests, and sanitizer_run.	Demo Stage 1 Gatekeeper, sandbox, and initial static/dynamic test runs on add.c.
Week 2 *(Oct 23 – Oct 29)*	Full Analyzer Suite & Parallel Engine	Complete compiler_warnings, static_analysis, function_split, edge_case_runner, complexity_estimator, similarity_flag, and llm_review; integrate parallel dispatch.	Demo all 9 analyzers running in parallel with automatic counter-testcase generation.
Week 3 *(Oct 30 – Nov 5)*	Streamlit UI, Reporting & Final Video	Build Streamlit ui.py and reporter.py; verify 100% score reproducibility; record 15–20 min YouTube demo and prepare final presentation.	Submit GitHub repo, final report, and YouTube demo video.